Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Max Lin

Summary. I evaluated 9,636 responses generated by 12 LLMs (from 6 families: GPT, Claude, Llama, Qwen, Mistral, Google) to the same 803 user prompts, trained a convolutional network (CNN) and a bidirectional LSTM (RNN) to predict the family of the model that generated the response. From the response only, the CNN achieves 56.2% accuracy and the BiLSTM 53.0% on unseen prompts (chance = 16.7%). The information about the prompt is not informative for distinguishing between the families (input-only = chance) because each prompt was answered by all of them. Moreover, adding the prompt to the response does not improve performance. A simulated data-collection bias shows how input-only classifiers can appear to have good performance on real-world datasets (RQ2).

1. Problem Definition and Dataset Curation

1.1 Task

Given a text produced by a chat LLM (possibly plus the user prompt that prompted it), predict which family of LLM generated it. Each example is a triple (LLM_name, LLM_input, LLM_output). We classify at the level of the family (6 classes) and not the exact model, because the task asks about families, and because two versions from the same vendor (e.g. GPT-4-Turbo, GPT-4o) are trained on similar data and post-training recipe, so their family is the natural unit of analysis of a "fingerprint". We also keep the exact model name, which we use to conduct a per-model analysis.

1.2 Data source and collection Generating thousands of our own responses using the paid APIs was not feasible, so we obtained the outputs from their publicly released evaluation set (Li et al., 2023; github.com/tatsu-lab/alpaca_eval, Apache-2.0 license). I call the leaderboard storing, for every evaluated model, its raw answer to the same 805 instructions (results//model_outputs.json). The instructions came from five public instruction sets (Self-Instruct, OpenAssistant, Koala, Anthropic helpful-base, Vicuna) covering advice, factual questions, creative writing, coding and math. The data does not contain any private or personal information as the instructions were written by researchers or crowd-workers for the purpose of public benchmarks.

Out of the ~230 entries on the leaderboard we took only the ones that are official vendor models and selected two models per family so that all families are represented more than once (Table 1). This is important because if every family was one model the classifier could memorize one model's quirks instead of a family fingerprint.

1.3 Cleaning, splitting and annotation

(1) I have discarded empty answers (1 by gemini-pro) and retained only those prompts to which all 12 models answered, giving 803 prompts × 12 = 9,636 examples (1,606 per family). The prompts are short (mean 29 words, median 18) and responses are on average 200–300 words. (2) I split by prompt (not row): 70/15/15% of the prompts (562/120/121) are used to train/validation/test along with all 12 of their answers (6,744/1,440/1,452 rows). A random row split would have put GPT's answer to a prompt in training set and Claude's answer to the same prompt in test set, allowing the model to use topic words. The prompt-based split guarantees that each test prompt is new. (3) we attached a task type to each prompt according to keyword rules (data/build_dataset.py): technical (code, math, data formats; 85 prompts), writing (stories, poems, emails, rewriting; 148) and general QA (all else: explanations, advice, facts; 570). We have manually validated a sample of the tags; they are not

perfect (some borderline cases get misclassified) but sufficient for a coarse cross-task evaluation.

Why this design. This design ensures that any difference that the classifier sees arises from differences in how the model writes rather than from differences in what it is asked about. This controlled design is what makes the RQ2 interpretable (Section 3.2). The trade-off is that the responses are from one benchmark and one time period (2023–2024 model versions) and that each vendor's default API settings are used.

2. CNN / RNN Implementation and Training

2.1 Text representation

I implemented the tokenizer instead of a standard word tokenizer since stylistic clues are embedded in characters that most tokenizers discard. The regular expression preserves words (case preserved, with contractions such as I'd as one token), numbers and each punctuation or markdown symbol as a separate token (, #, -, :, ...). Line breaks are transformed into a special token, so that a bulleted list will be tokenized as - for instance. The vocabulary is built from the training split only (tokens with a frequency ≥ 2, ~28,985 types) and unseen tokens during training are mapped to during evaluation. Responses are truncated to 512 tokens (22% of responses were longer) and prompts to 128 tokens. The three input settings of RQ2 are: output only; input only; and input + output = prompt tokens, a token followed by the response tokens.

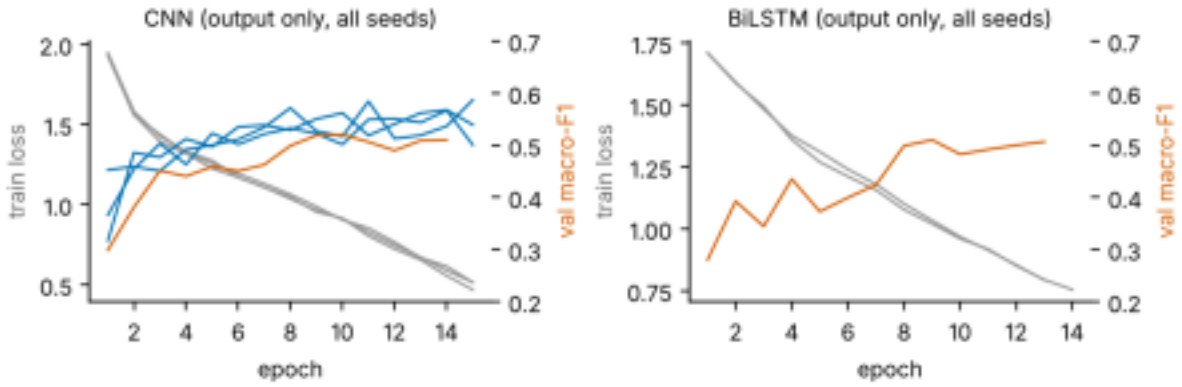
2.2 Models

CNN (Kim, 2014). A 128-d embedding layer, trained from scratch, followed by three parallel 1-D convolutions over width 3, 4 and 5 with 128 filters each (ReLU). Each feature map is max-pooled over time (masking out padding positions) to give a 384-d vector which is then dropped out (0.5) and passed through a linear layer to 6 logits. Each filter acts like a learned n-gram detector (e.g. 1 . or What a great question) and the max-pooling asks of each such pattern, does it appear anywhere in the response?

RNN: bidirectional LSTM. The same 128-d embedding (dropout 0.3), a single-layer bidirectional LSTM of 128 hidden units per direction, and a sentence vector of masked max-pool and masked mean-pool of all hidden states (512-d) followed by dropout and linear. We first tried taking the final hidden states with pack_padded_sequence but on our CPU-only setup, a batch with mixed lengths took ~30 s versus ~1 s unpacked, so we run the LSTM on padded batches and use pooling with masks rather than the final state. Batches are bucketed by length so there is very little padding.

2.3 Training details

Loss is cross-entropy. After every epoch we evaluate macro-F1 on the validation prompts and keep the best checkpoint; training stops when validation F1 has not improved for 4 epochs. Test prompts are used only once, for the final numbers. We report the mean ± standard deviation over random seeds (3 for the CNN, 2 for the slower LSTM). Everything ran on a 2-core CPU without a GPU, which is why we kept the models small and the number of LSTM seeds low. Hyper-parameters were set once from common defaults and a single trial run, not tuned on the test set. As a non-neural reference point (not required) we also trained a TF-IDF (word 1–2-grams + character 2–5-grams) logistic-regression model on the same splits.



3. Results (RQ1 and RQ2)

Output only (RQ1) 56.2 ± 0.2 0.554 53.0 ± 0.9 0.518 60.5 Input only 16.7 ± 0.0 0.104 16.7 ± 0.0 0.105 16.7 Input + Output 56.6 ± 0.4 0.562 52.4 ± 1.4 0.504 59.4

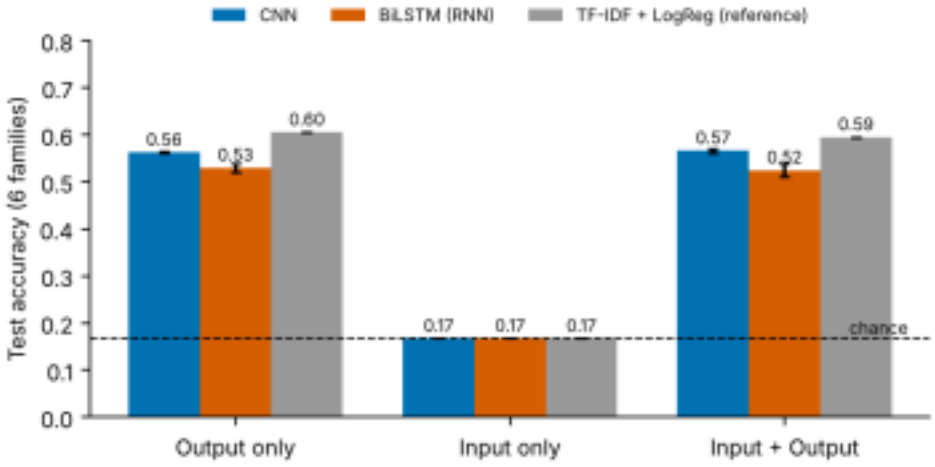
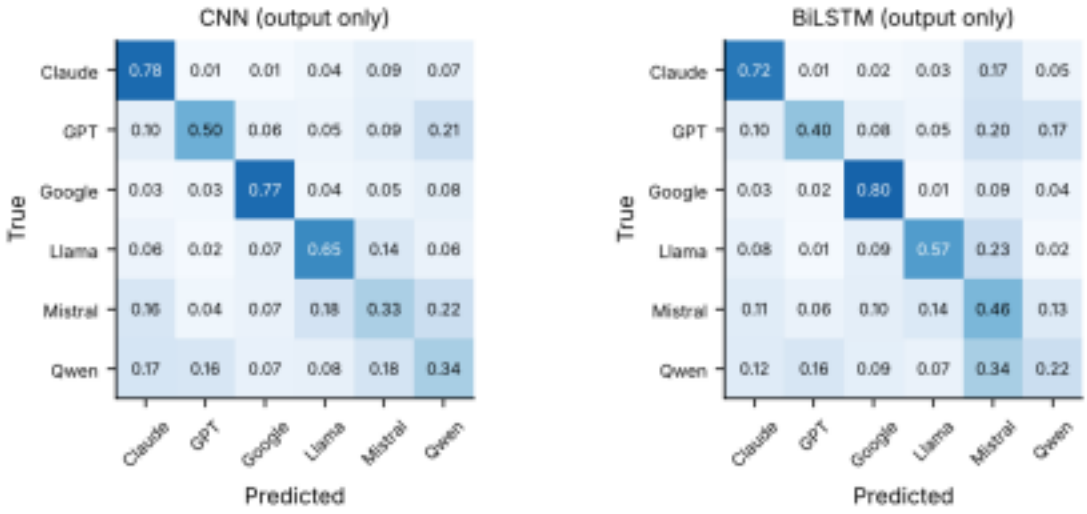


Figure 2. Test accuracy by input setting. Error bars: std over seeds. The dashed line is chance (1/6).

3.1 RQ1: Can we identify which LLM generated a response?

Yes, and well above chance. Just from the response, the CNN correctly identifies the family of an answer to a never-seen prompt 56.2% of the time (macro-F1 0.554) and the BiLSTM 53.0% (macro-F1 0.518) versus 16.7% for guessing. The CNN is 3.2 points better and much more stable across seeds (std 0.2 vs. 0.9). We think the CNN fits the task better because evidence is local and position-independent: a bold heading, an opening phrase, a list marker (all of which are max-pooled n-gram filters) versus hundreds of steps for the LSTM and harder to optimize from scratch on ~6.7k examples. The TF-IDF reference hits 60.5%, so a good chunk of the signal is present as surface n-grams; the neural models don't find much beyond that with this amount of data.



The families of the models cannot be considered entirely equal. According to the indicators presented in Figure 3, the CNN recall reaches 77.7% (Claude), 77.0% (Google), 34.0% (Qwen), and 33.5% (Mistral). The way Google models are described is unique in that many of them do not have an introductory sentence but rather start listing answers right away or provide them in one sentence with bold headings; Claude 3 rarely uses bold markings in its responses. Qwen and Mistral are the most challenging models to differentiate between due to their similar response structures, with both tending to sound like an “average” assistant: a short introduction, numbered list, and a closing sentence. They are frequently confused with Claude, Mistral with Llama, and Qwen with GPT-3.5. One possible explanation is that open models were trained on dialogues produced by or imitating GPT, resulting in their style being conflated.

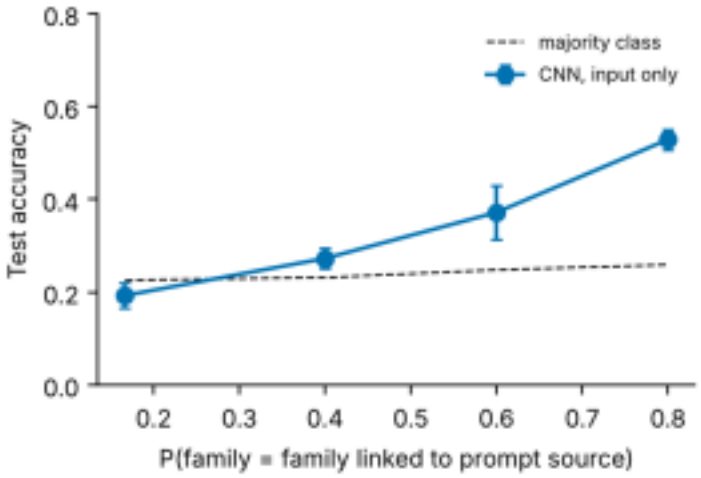
3.2 RQ2: Does the user's prompt help identify the LLM?

(i) **Can the LLM be predicted from the prompt alone?** In the data, classifying the input-only yields a score of 16.7% for the input-only CNN, and 16.7 for the BiLSTM, i.e., chance performance. This is by design: As each prompt appears 12 times in the dataset, we cannot predict any meaningful label from the prompt. Consequently, the two networks have correctly learned to output roughly the same distribution for all prompts. We consider this one of the most important sanity checks on the dataset: If input-only classification had performed above chance, either our split or our data generation has a bug.

(ii) Does adding the input improve classification? Not meaningfully.

Input plus output gives 56.6% (CNN) and 52.4% (BiLSTM), compared with 56.2% and 53.0% for output only; TF-IDF is also not improved (59.4% vs. 60.5%). The changes are small and both positive and negative, i.e., within run-to-run noise. In principle the prompt could help as context (e.g. knowing a one-line answer was given to a request for a long essay), but with our data size the models cannot learn such interactions, and the extra prompt tokens mostly add noise.

(iii) When can input-only classification work surprisingly well? Only when the dataset itself couples prompts to models. That is easy in practice: if one person collects GPT answers to coding questions and another collects Llama answers to chit-chat, then a classifier can learn "coding prompt → GPT" without ever seeing the writing. To demonstrate this we made biased versions of our data, in which for every prompt we keep a single response. We trained the input-only CNN on each version.



With no link (p = 1/6) the prompt-only model has 19.3% accuracy, just above chance. As the collection bias increases, the same model has 52.9% accuracy at p = 0.8 (majority-class baseline of 25.9%) without having seen a single prompt written by an LLM. The various prompt sources have different styles (Self-Instruct prompts are templated tasks, Koala prompts are casual chat, etc.), so the network learns the source and through the source the label. A high input-only accuracy is a warning sign for dataset bias, not evidence of an LLM fingerprint. This is why we used a dataset where each family answers each prompt.

5. Lessons and Experience

The split matters more than the model. A random row split is the default in most tutorials. With 12 answers per prompt it would have put most of the test prompts into training under other labels and let topic words leak. Splitting by prompt was a minor code change but determines what the reported number actually means. The input-only result (exactly chance) serves as the sanity check that the split is correct.

Dataset design answers research questions before any training takes place. A dataset where every family is asked the same prompts made RQ2 almost trivially answerable for our data, and the bias simulation showed us how an uncontrolled dataset can lead to an impressive but meaningless input-only score. When we hear about "AI text detectors" that have very high accuracy, the first question we ask ourselves is now about the data collection process.

Tokenization is a modeling decision. A standard tokenizer would have stripped away most of the signal: markdown symbols, list markers and line breaks are precisely where the families are most visibly different.

A simple baseline keeps you honest. The TF-IDF + logistic regression model was trained in about a minute and was at least as accurate as both the neural networks. The CNN and LSTM have to learn word embeddings from scratch, without any pre-trained embeddings, and with only ~6,700 training examples, while n-gram counts encode the surface-level patterns directly. A pre-trained transformer would probably be a better choice, but we stuck to the required architecture and a CPU-friendly budget.

CNN vs. RNN in practice. The CNN was faster (≈57 s vs. ≈84 s per epoch on one CPU thread), more stable across seeds and more accurate. Our LSTM was also hampered by an engineering limitation: packed sequences were about 30x slower on our CPU build and we had to revert to using padded, length-bucketed batches with masked pooling. It would have saved us an hour-long training run if we had profiled one batch first before proceeding to train for dozens of hours.

Limitations. (i) All responses were generated using one model (the default) and one particular temperature value without additional system prompts; in practice, users would be using different temperature values and system prompts that suppress formatting instructions. (ii) We evaluated only two models per family, while to claim a “family”, one would need to evaluate at least three models (e.g., Claude 3.5 Sonnet is already different from Claude 3 Opus in our per-model evaluations). (iii) Due to the computational budget, we only evaluated three seeds for the CNN and two seeds for the BiLSTM, and did not perform a hyper-parameter search.

References

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP.

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.

Li, X., Zhang, T., Dubois, Y., et al. (2023). AlpacaEval: An Automatic Evaluator of Instruction-following Models. github.com/tatsu-lab/alpaca_eval.

Monroe, B., Colaresi, M., & Quinn, K. (2008). Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict. Political Analysis.

Covington, M. & McFall, J. (2010). Cutting the Gordian Knot: The Moving-Average Type-Token Ratio (MATTR). Journal of Quantitative Linguistics.